Technical Support Assistant – Troubleshooting Knowledge Base

Knowledge document for RAG-based technical support chatbot

This document provides technical troubleshooting knowledge for a support assistant. It is designed to help the RAG chatbot answer common technical questions related to databases, frontend applications, backend services, APIs, integrations, authentication, configuration, and overall system behavior. The assistant should use the relevant section to identify likely causes, recommend safe troubleshooting steps, and explain when an issue should be escalated to a technical support or engineering team.

1. Database Troubleshooting

Database troubleshooting covers problems where an application cannot connect to the database, queries fail, records are missing or incorrect, database operations are slow, or database changes are not reflected in the application. Common symptoms include connection refused errors, authentication failures, timeout errors, SQL errors, duplicate records, missing records, transaction failures, and unexpectedly slow responses.

Database Connection Problems

When an application cannot connect to a database, first verify that the database server is running and reachable. Check the configured database host, port, database name, username, and password. Confirm that the application is using the expected environment or configuration file. A wrong port, incorrect hostname, expired credentials, disabled database service, or network restriction can prevent a connection.

Database Query Errors

For SQL query failures, inspect the exact database error message and identify whether the problem is related to syntax, missing tables, missing columns, invalid data types, constraints, permissions, or transaction state. Confirm that database migrations have been applied when the application expects a recently added table or column.

Missing or Incorrect Data

When expected records are missing, verify that the correct database, schema, environment, and account are being used. Check query filters, record status, transaction completion, and whether the write operation actually succeeded. For incorrect values, compare the stored record with the request received by the backend and review application logs.

Database Performance

Slow database operations may be caused by inefficient queries, large result sets, missing indexes, locking, high connection usage, or resource limitations. Check query execution time, database CPU and memory usage, connection counts, and slow-query logs. Avoid changing production indexes or deleting data without appropriate approval.

2. Frontend Troubleshooting

Frontend troubleshooting covers problems in the user interface, browser behavior, page loading, form submission, JavaScript execution, state management, responsive behavior, and communication with backend APIs. Common symptoms include blank pages, broken layouts, buttons not responding, incorrect data display, browser console errors, failed API requests, and unexpected UI behavior.

Page or Application Not Loading

Check whether the frontend development or production server is running and whether the browser can reach the application URL. Inspect the browser console and network panel for JavaScript, routing, asset-loading, CORS, or HTTP errors. Clear stale browser cache when an old frontend bundle may be loaded.

UI and Component Problems

If a button, form, modal, or component does not work, verify that the expected event handler is connected and that the component receives the required data. Check browser console errors, component state, conditional rendering, and form validation. For React applications, verify state updates, props, effects, and component lifecycle behavior.

Frontend API Communication

When the frontend cannot retrieve or submit data, inspect the browser Network tab. Verify the request URL, HTTP method, headers, request body, authentication token, and response status. A 400 response generally indicates a request or validation problem, a 401 or 403 indicates authentication or authorization problems, and a 404 may indicate an incorrect endpoint.

3. Backend Troubleshooting

Backend troubleshooting covers server-side application logic, request processing, database operations, validation, business logic, background processing, exceptions, and server configuration. Common symptoms include HTTP 500 errors, incorrect API responses, failed database operations, slow requests, application crashes, and unexpected server behavior.

Server Errors

For a server error, check the backend logs and identify the exception, request path, timestamp, and affected operation. Do not assume that every 500 response is a database problem. The cause may be application logic, invalid assumptions about input data, missing configuration, dependency errors, or an external service failure.

Request Validation

If the backend rejects a request, verify required fields, field names, data types, allowed values, file formats, request size, and authentication requirements. Compare the actual request payload with the API contract. Validation errors should be returned clearly so the frontend or client can correct the request.

Backend Performance

Slow backend responses can result from database queries, external API calls, expensive processing, large files, inefficient code, or insufficient server resources. Measure where time is spent before changing the implementation. Review request logs, database timings, external service timings, and resource utilization.

4. API Troubleshooting

API troubleshooting covers problems involving REST or other service endpoints, including incorrect URLs, HTTP methods, request payloads, headers, authentication, response formats, status codes, timeouts, and server-side validation.

Endpoint and Method Problems

Confirm that the client is calling the correct endpoint and HTTP method. GET is commonly used to retrieve information, POST to create or process information, PUT or PATCH to update information, and DELETE to remove information. The exact contract of the application should always be followed.

Request and Response Problems

Verify Content-Type, Authorization, Accept headers, query parameters, path parameters, and request body. Confirm that JSON is valid when JSON is expected. For file uploads, verify multipart form-data and the expected field name. Inspect the response body because it often contains a useful validation or error message.

Common HTTP Status Codes

A 200 response normally indicates a successful request. A 201 commonly indicates successful creation. A 400 indicates a malformed or invalid request. A 401 indicates that authentication is required or the supplied credentials are invalid. A 403 indicates that access is forbidden. A 404 indicates that the requested resource or endpoint was not found. A 409 commonly indicates a conflict. A 422 may indicate validation failure depending on the API. A 429 indicates rate limiting. A 500 indicates an unexpected server-side error, while 502, 503, and 504 commonly indicate gateway, availability, or upstream timeout problems.

5. Integration Troubleshooting

Integration troubleshooting covers communication between the application and external services, third-party APIs, payment services, messaging systems, cloud services, identity providers, and other applications. Integration failures can occur because of invalid credentials, incorrect endpoints, incompatible request formats, expired tokens, network failures, service outages, rate limits, or changed third-party API contracts.

External Service Connection

Verify the external service URL, credentials, API version, required headers, and network access. Check whether the service is available and whether the integration is using the correct environment, such as development, staging, or production.

Data Mapping Problems

When two systems exchange data incorrectly, compare the outgoing request with the receiving system's expected schema. Check field names, required fields, data types, date formats, identifiers, and nested objects. A successful HTTP response does not always mean that the business operation completed correctly.

Integration Timeouts and Retries

Timeouts can occur when an external service is slow or unreachable. Review timeout settings and logs. Retries should be used carefully because repeating non-idempotent operations can create duplicate actions. Where supported, use request identifiers or idempotency mechanisms to prevent duplicate processing.

6. Authentication Issues

Authentication troubleshooting covers login failures, expired sessions, invalid credentials, access tokens, refresh tokens, password problems, account lockouts, and identity-provider communication. Authentication confirms who the user is, while authorization determines what the authenticated user is permitted to access.

Login Failure

For login failures, verify the username or email, password, account status, authentication endpoint, and request format. Check whether the account is active, locked, or required to complete another verification step. Avoid exposing passwords or sensitive authentication information in logs.

Token Problems

If an authenticated API request returns 401, verify that the access token exists, is correctly formatted, has not expired, and is sent in the required Authorization header. If refresh tokens are used, verify the refresh flow and token storage rules. If a token is valid but access is denied, investigate authorization and permissions rather than authentication.

Role and Permission Problems

A user may successfully log in but still be unable to access a feature. Check the user's assigned role, permissions, resource ownership, and backend authorization rules. For role-based systems, confirm that the frontend restrictions are consistent with backend enforcement.

7. Configuration Issues

Configuration troubleshooting covers environment variables, application settings, database settings, API URLs, ports, credentials, feature flags, file paths, CORS settings, logging configuration, and deployment-specific values.

Environment Variables

Verify that required environment variables exist and contain the correct values for the current environment. Common examples include database connection values, API keys, secret

keys, service URLs, allowed origins, and application environment settings. Never expose secret keys, passwords, or tokens in source code or support responses.

Environment Mismatch

An application can appear broken when it is connected to the wrong environment. Confirm whether the application is using development, testing, staging, or production configuration. Check API base URLs, database names, storage locations, and external service endpoints.

CORS and Network Configuration

If a browser blocks a frontend request because of CORS, verify that the backend allows the expected frontend origin, protocol, and port. CORS configuration should be reviewed on the backend rather than solved by disabling browser security. Also check firewalls, proxy settings, DNS, and network access when the service is unreachable.

8. System Troubleshooting

System troubleshooting covers issues affecting the overall application environment, including server availability, CPU and memory usage, disk space, processes, network connectivity, operating-system errors, service failures, file permissions, and resource exhaustion.

Application or Service Unavailable

Check whether the required service or process is running. Review application and system logs around the time of failure. Confirm that the expected port is listening and that the server has sufficient resources. If a service repeatedly stops, identify the underlying error rather than repeatedly restarting it without investigation.

CPU and Memory Problems

High CPU or memory usage can cause slow responses, process termination, or system instability. Identify which process is consuming resources and check whether the usage is temporary or sustained. Review recent deployments, traffic changes, background jobs, memory leaks, and unusually large requests.

Disk and File-System Problems

Low disk space can prevent applications from writing logs, temporary files, uploaded documents, or database data. Check available disk space and identify large files before removing anything. Verify file permissions when an application reports that it cannot read, create, modify, or delete a required file.

Network Connectivity

For connectivity problems, verify DNS resolution, network access, firewall rules, proxy configuration, required ports, and service availability. Compare whether the issue affects one client or multiple users to determine whether it is likely local or system-wide.

9. General Troubleshooting Workflow

The support assistant should first understand the user's exact symptom and identify the affected component. It should then collect useful information such as the error message, HTTP status code, affected URL or feature, time of occurrence, environment, and whether the issue affects one user or multiple users. The assistant should avoid guessing when critical information is missing.

The assistant should identify likely causes from the available knowledge, recommend safe diagnostic steps, and explain the expected result of each step. If a step resolves the issue, the assistant can provide a short confirmation. If the problem continues, it should request the next relevant piece of information or recommend escalation to the appropriate technical team.

For production systems, troubleshooting instructions should prioritize data safety and service stability. The assistant should not recommend destructive database commands, credential disclosure, disabling security controls, or unapproved production changes. Sensitive values such as passwords, API keys, access tokens, database credentials, and private customer information should not be requested unnecessarily.